

# Journaling File Systems

## 1 Basic Idea

### Journaling File System

A **journaling file system** writes a log record of planned file-system changes before applying those changes to the main file system. If the system crashes, the log tells the operating system what operation was in progress.

- Journaling is used to improve file-system reliability after crashes.
- The file system first records what it is about to do.
- Only after the log entry is safely written does it modify the real file-system structures.
- After the operation finishes successfully, the log entry can be erased or marked complete.
- During reboot, the system checks the journal and completes any unfinished operations.

## 2 Why Journaling Is Needed

### Crash Consistency Problem

Many file-system operations require several separate updates. If the system crashes after some updates but before others, the file system can become inconsistent.

- A disk update is not usually a single all-or-nothing action.
- File metadata and file data may be stored in different disk blocks.
- Directory entries, i-nodes, free-block maps, and free-i-node lists may all need updates.
- A crash in the middle can leave some structures updated and others old.
- Journaling gives the recovery code a clear record of what must be completed.

## 3 Example: Removing a File

### UNIX File Removal

Removing a file in a UNIX-style file system requires several metadata updates.

1. Remove the file's entry from its directory.
2. Release the file's i-node to the pool of free i-nodes.
3. Return the file's disk blocks to the pool of free disk blocks.

- If all three steps complete, the file is removed correctly.
- If a crash happens between steps, the order matters.
- Different crash points can create different kinds of corruption or resource loss.

## 4 Crash Cases Without Journaling

| Crash Point                                       | Resulting Problem                                                               |
|---------------------------------------------------|---------------------------------------------------------------------------------|
| After removing directory entry only               | The i-node and blocks are unreachable but not free, so disk resources are lost. |
| After removing directory entry and freeing i-node | The data blocks may remain lost.                                                |
| After freeing i-node first                        | The old directory entry may point to an i-node later reused by another file.    |
| After freeing blocks first                        | A valid i-node may point to blocks that are already in the free pool.           |

### Worst Case

If blocks are returned to the free pool before the i-node is cleared, two or more files may later point to the same disk blocks.

## 5 How Journaling Handles the Operation

### Journal First

The key rule is: write the plan to the journal before changing the actual file-system structures.

1. Write a journal entry describing all required actions.
2. Force the journal entry to disk.
3. Optionally read it back to verify that it was written correctly.
4. Perform the real file-system updates.
5. After all updates finish, erase or mark the journal entry complete.

## 6 Recovery After a Crash

- During reboot, the file system checks the journal.
- If no incomplete operation is recorded, normal mounting can continue.
- If a pending operation is found, the file system reruns the logged actions.
- The actions may be repeated after repeated crashes.
- When the operation is finally complete, the journal entry is cleared.

## 7 Idempotent Operations

### Idempotent Operation

An operation is **idempotent** if it can be repeated many times and still produces the same final state without causing harm.

- Journaling depends on logged operations being idempotent.
- Recovery may repeat an operation that was already partly completed.
- Repeating the operation must not damage the file system.
- File-system data structures must be designed so journal actions are safe to replay.

| Operation                                                | Idempotent?                                                  |
|----------------------------------------------------------|--------------------------------------------------------------|
| Mark i-node $k$ as free in a bitmap                      | Yes, setting the same bit repeatedly is harmless.            |
| Mark block $n$ as free in a bitmap                       | Yes, the final bitmap state is unchanged by repetition.      |
| Remove any directory entry named foobar                  | Yes, once it is gone, removing it again has no extra effect. |
| Append block $n$ to the end of a free list               | No, it may add the same block more than once.                |
| Add block $n$ only if it is not already on the free list | Yes, but it is more expensive.                               |

## 8 Atomic Transactions

### Atomic Transaction

An **atomic transaction** groups several file-system actions so the system completes either all of them or none of them.

- A transaction begins with a begin-transaction operation.
- It ends with an end-transaction operation.
- The file system treats all operations inside the transaction as one logical unit.
- After a crash, recovery must not leave only part of the transaction applied.
- Atomic transactions make crash recovery more systematic.

## 9 Examples of Journaling File Systems

- NTFS uses an extensive journaling system.
- Linux ext3 added journaling while remaining compatible with ext2.
- ReiserFS was an early Linux journaling file system.
- OS X provides journaling as an option for supported file systems.

## 10 Journaling vs Log-Structured File Systems

| Point         | Journaling File System                | Log-Structured File System           |
|---------------|---------------------------------------|--------------------------------------|
| Main purpose  | Crash recovery and consistency        | High write throughput                |
| Log role      | Records planned metadata changes      | Stores the main file-system contents |
| Compatibility | Easier to add to existing designs     | Often requires a different layout    |
| Recovery      | Replay or finish logged operations    | Use log and cleaner metadata         |
| Common use    | Widely used in practical file systems | Less widely used directly            |

## 11 Key Points

- Journaling protects the file system from inconsistent partial updates.
- The journal records the intended operation before the real update happens.
- Crash recovery checks the journal and completes unfinished work.
- Logged operations must be idempotent so they can be safely repeated.
- Atomic transactions group related updates into all-or-nothing units.

### Final Point

Journaling file systems make crash recovery fast and reliable by recording file-system intentions before applying them and replaying safe, idempotent operations after a failure.